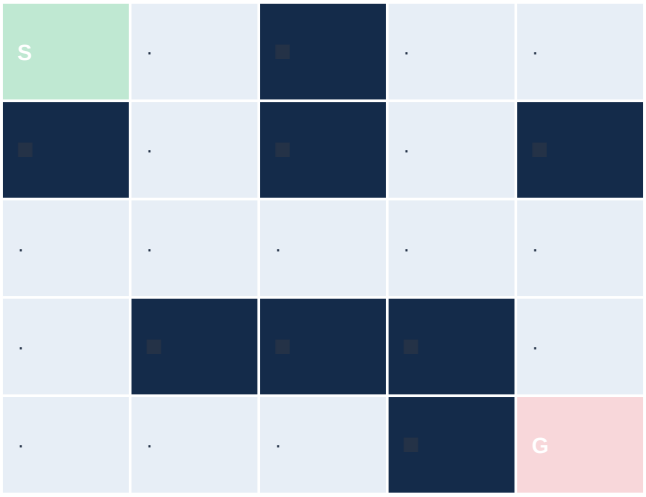
COMPUTER LABORATORY – 1

Q-Learning in a Maze Environment

Detailed code walkthrough • Reinforcement Learning theory • Viva questions & cross-questions



Based on your uploaded Assignment-11.ipynb
The notebook implements a small 5 × 5 maze, trains an agent using tabular Q-learning, and tests the learned greedy route.

Prepared as a beginner-friendly viva and practical-exam study guide

1. Understand the experiment

What is the aim? The aim is to teach an agent to move through a maze from a start cell to a goal cell. Some cells are blocked by walls. The agent does not receive a pre-written route. It tries actions, receives rewards or penalties, and gradually learns which action is useful in each position.

The assignment list describes this as implementing Reinforcement Learning with a maze environment in which the agent needs to explore. Your notebook uses Q-learning, a model-free reinforcement-learning algorithm.

Important terms

Term	Simple meaning in this experiment
Agent	The learner that moves around the maze.
Environment	The 5 × 5 maze and its rules.
State	The agent's current cell, represented by (row, column).
Action	One of four moves: up, down, left, or right.
Reward	A numerical feedback value after an action.
Policy	The rule used to choose an action in a state.
Episode	One training attempt, starting at S and ending at the goal or move limit.
Q-value	The learned estimate of how good an action is in a particular state.

Maze representation

The NumPy array uses 0 for an open cell and 1 for a wall. The start is (0, 0), at the top-left. The goal is (4, 4), at the bottom-right. Coordinates are zero-indexed: the first row/column is numbered 0, not 1.

Remember: A state is a position, not the entire path taken to reach that position. The Q-table stores four action values for each of the 25 grid positions.

2. Code walkthrough – setup and maze

Imports

```
import numpy as np
import matplotlib.pyplot as plt
```

numpy (np): Used for arrays, random-number generation, and numerical operations. **matplotlib.pyplot (plt):** Used to draw the maze and the route after training.

Random generator

```
rng = np.random.default_rng(42)
```

Creates a NumPy random-number generator. The seed 42 makes the random sequence repeatable, so the experiment is easier to reproduce. It does not make the learning process intelligent; it only controls randomness.

Creating the maze

```
maze = np.array([[0, 0, 1, 0, 0],
                 [1, 0, 1, 0, 1],
                 [0, 0, 0, 0, 0],
                 [0, 1, 1, 1, 0],
                 [0, 0, 0, 1, 0]])
```

This is a 2D, 5-row by 5-column array. For example, maze[0, 2] is 1, so that cell is a wall. maze[2, 3] is 0, so that cell is open. In Python, array indexing starts at zero.

Start, goal, actions, and Q-table

```
start, goal = (0, 0), (4, 4)
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
Q = np.zeros((5, 5, 4))
alpha, gamma = 0.2, 0.95
```

Each action is a change in (row, column): up decreases row; down increases row; left decreases column; right increases column. Q has shape (5, 5, 4): 5 rows × 5 columns × 4 actions. It starts with zero estimates for every state-action pair.

Variable	Meaning
alpha = 0.2	Learning rate: how strongly a new estimate changes the old Q-value.
gamma = 0.95	Discount factor: importance given to future rewards.
Q	Table of estimated action values.
rng	Repeatable random generator.

Potential viva cross-question: Why initialise Q with zeros? Because no action has been learned to be better at the beginning. Zero is a simple neutral initial estimate; other initialisation methods are possible.

3. The step() function – environment rules

This function receives the current position and an action index. It calculates the proposed next cell, checks whether the move is legal, and returns the result.

```
def step(position, action):
    row = position[0] + actions[action][0]
    column = position[1] + actions[action][1]
    if not (0 <= row < 5 and 0 <= column < 5) or maze[row, column] == 1:
        return position, -0.5, False
    next_position = (row, column)
    done = next_position == goal
    return next_position, 10 if done else -0.1, done
```

Line-by-line explanation

• **position[0]** is the current row and **position[1]** is the current column.

• **actions[action]** selects the movement vector. If action is 0, the vector is (-1, 0), meaning move up.

• The new row and column are calculated by adding the movement vector to the current position.

• The condition checks two kinds of invalid move: going outside the 5 × 5 grid, or entering a wall (maze value 1).

• For an invalid move, the agent stays in the same cell, receives -0.5, and done is False.

• For a valid move, next_position is created as a tuple. done becomes True only if this new position equals the goal.

• The function returns three values: next position, reward, and whether the episode is finished.

Why are there three return values?

The training loop needs the next state to update Q, the reward to calculate the learning target, and done to know when to stop the episode. Python allows a function to return multiple values as a tuple, which can be unpacked into three variables.

Reward design

Situation	Reward	Reason
Hit a wall / leave grid	-0.5	Discourages invalid moves.
Valid move, not at goal	-0.1	Small step penalty encourages shorter routes.
Reach the goal	+10	Strongly encourages reaching the destination.

Cross-question: Does a negative reward mean the agent is doing something wrong? Not necessarily. It is feedback designed by the programmer. Here, small negative rewards encourage the agent to reach the goal efficiently.

4. Q-learning – theory behind the training

Reinforcement Learning (RL)

Reinforcement Learning is a machine-learning approach in which an agent learns by interacting with an environment. It takes an action, observes the result, receives a reward, and improves its future decisions. Unlike supervised learning, the training data is not simply a fixed set of input-output labels.

What is Q-learning?

Q-learning is a model-free, off-policy, value-based reinforcement-learning algorithm. It learns a value Q(s, a): how useful it is to take action a while in state s, considering the reward now and possible future rewards.

Q-learning update equation

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \max Q(s',a') - Q(s,a)\,]$$

Symbol	Meaning
s	Current state (current cell).
a	Action selected in the current state.
r	Immediate reward received after the action.
s′	Next state reached after the action.
α (alpha)	Learning rate; controls the size of the update.
γ (gamma)	Discount factor; controls the value of future rewards.
max Q(s′, a′)	Best estimated action value in the next state.

In the notebook, this equation is implemented as:

```
target = reward if done else reward + gamma * np.max(Q[next_position])
Q[position][action] += alpha * (target - Q[position][action])
```

The target is the immediate reward plus discounted best future value. If the goal has been reached, there is no future step to plan for, so the target is only the reward. The difference (target – old Q-value) is the temporal-difference error. Multiplying it by alpha makes a partial correction rather than replacing the old value completely.

Why is it called model-free?

The algorithm does not need a complete mathematical model of all maze transitions in advance. It learns from observed transitions (state, action, reward, next state). In this notebook, the step() function still defines the environment rules; model-free means Q-learning does not need to be given a transition/reward model as its planning input.

5. Training loop – how learning happens

```
for episode in range(2000):
    position = start
    epi = max(0.05, 0.9 * (0.997 ** episode))
    for move in range(100):
        if rng.random() < epi:
            action = int(rng.integers(4))
        else:
            action = int(np.argmax(Q[position]))

        next_position, reward, done = step(position, action)
        target = reward if done else reward + gamma * np.max(Q[next_position])
        Q[position][action] += alpha * (target - Q[position][action])
        position = next_position
        if done:
            break
```

Outer loop: episodes

range(2000) runs training for 2,000 episodes. At the start of every episode, position is reset to start. This gives the agent repeated opportunities to learn from the same maze.

Inner loop: moves

range(100) allows at most 100 moves in one episode. The limit prevents an episode from continuing forever if the agent keeps wandering or gets stuck.

Exploration vs exploitation

The variable epi is the epsilon value in an epsilon-greedy action-selection strategy. With probability epsilon, the agent explores by selecting a random action. Otherwise, it exploits by selecting the action with the largest learned Q-value.

$$\textbf{epsilon} = \max(0.05,\ 0.9 \times 0.997^{\text{episode}})$$

At the beginning epsilon is 0.9, so random exploration is frequent. It gradually decreases as episodes increase, but never goes below 0.05. The minimum keeps a small amount of exploration during training.

Action selection functions

Code	What it does
rng.random()	Generates a random decimal in [0, 1).
rng.integers(4)	Generates one of 0, 1, 2, or 3.
np.argmax(Q[position])	Returns the index of the largest of the four Q-values for the current cell.
int(...)	Converts the selected NumPy integer to a regular Python integer.

After selecting an action, the code calls step(), calculates the target, updates one Q-value, and moves the agent to next_position. If done is True, break ends the current episode early because the goal was reached.

6. Testing the learned policy and plotting

```
position = start
path = [position]
for move in range(50):
    action = int(np.argmax(Q[position]))
    position, reward, done = step(position, action)
    path.append(position)
    if done:
        break

print('Reached goal:', position == goal)
print('Learned path:', path)
```

This section tests the trained Q-table. It does not use random exploration. At every position, it chooses the action with the highest Q-value. This is called a greedy policy. The path list stores every visited position, beginning with the start.

Why is the test limit 50?

It prevents the test from running forever if the learned policy enters a loop. If the goal is not reached within 50 moves, the output reports False. This is a safeguard, not proof that the agent has failed permanently.

Visualisation

```
plt.imshow(maze, cmap='Greys', vmin=0, vmax=1)
plt.plot([p[1] for p in path], [p[0] for p in path],
         'o-', color='royalblue')
plt.text(0, 0, 'S', color='green', ha='center', va='center')
plt.text(4, 4, 'G', color='red', ha='center', va='center')
plt.xticks(range(5))
plt.yticks(range(5))
plt.title('Maze: route learned by Q-learning')
plt.show()
```

Function / argument	Purpose
plt.imshow()	Displays the maze array as an image.
cmap='Greys'	Uses a grey colour map for the grid.
vmin=0, vmax=1	Fixes the colour scale to the maze values 0 and 1.
plt.plot()	Draws the route as connected points.
[p[1] for p in path]	Uses column values for the horizontal (x) axis.
[p[0] for p in path]	Uses row values for the vertical (y) axis.
plt.text()	Places S and G labels on the start and goal cells.
plt.xticks(), plt.yticks()	Shows integer grid coordinates.
plt.title() / plt.show()	Adds a title and displays the final figure.

Common plotting cross-question: Why does x use p[1] and y use p[0]? A grid position is stored as (row, column), while a plot expects (x, y) = (column, row).

7. Viva questions – core theory

1. What is Artificial Intelligence?
AI is the field of building systems that perform tasks that normally require human-like intelligence, such as learning, reasoning, or decision-making.

2. What is Machine Learning?
A branch of AI in which systems learn patterns from data or experience to make predictions or decisions.

3. What is Reinforcement Learning?
A learning approach where an agent interacts with an environment and learns a strategy using rewards and penalties.

4. What are the main elements of RL?
Agent, environment, state, action, reward, and policy. Value functions are also important for evaluating future outcomes.

5. What is a state?
A description of the situation the agent is currently in. Here, it is the current (row, column) cell.

6. What is an action?
A choice the agent can make in a state. Here, up, down, left, or right.

7. What is a reward function?
The rule that assigns a numerical reward to an outcome. It tells the agent which outcomes are desirable.

8. What is a policy?
A strategy that maps a state to an action. A greedy policy selects the action with the highest Q-value.

9. What is a Q-table?
A table containing a value for every state-action pair. Here it has dimensions 5 × 5 × 4.

10. What does model-free mean?
The learner does not need a pre-supplied transition model to plan. It learns value estimates from experience.

11. What is the difference between exploration and exploitation?
Exploration tries actions to gain information. Exploitation chooses the action currently believed to be best.

12. What is epsilon-greedy?
A strategy that explores randomly with probability epsilon and otherwise selects the best-known action.

13. Why does epsilon decrease?
Early training needs broad exploration. Later training can rely more on learned values while retaining a small exploration chance.

14. What is the learning rate?
Alpha controls how much each new learning target changes the existing Q-value.

15. What is the discount factor?
Gamma controls how much future rewards matter compared with immediate rewards.

16. What is the difference between supervised and reinforcement learning?
Supervised learning uses labelled examples. RL learns through interaction and reward feedback, often without a correct action label for every state.

17. What is an episode?
One complete run of interaction, usually from an initial state until a terminal condition or step limit.

18. Why reward the goal with +10?
It gives a strong positive signal that reaching the destination is desirable.

19. Why penalise each ordinary move?
A small step cost encourages shorter solutions instead of wandering unnecessarily.

20. What is a terminal state?
A state that ends the episode. In this maze, reaching the goal is terminal.

8. Viva questions – code, functions and tricky cross-questions

1. Why use NumPy instead of a normal Python list?
NumPy provides efficient multidimensional arrays and convenient numerical operations such as zeros(), max(), and random generation.

2. What does np.zeros((5, 5, 4)) create?
A 3D array of shape (5, 5, 4), filled with zeros. It stores four action values for each of 25 grid cells.

3. What does np.argmax() return?
The index of the maximum value, not the maximum value itself. That index corresponds to an action number from 0 to 3.

4. What if two actions have equal Q-values?
np.argmax returns the first maximum index. With all-zero initial values, greedy selection initially favours action index 0 unless exploration selects another action.

5. Why convert the action to int?
NumPy random and array operations may return NumPy integer types. int() converts the result to a standard Python integer.

6. What does rng.default_rng(42) do?
Creates a modern NumPy random generator with a fixed seed. Re-running with the same code and environment gives reproducible random choices.

7. Why is step() a separate function?
It keeps environment transition rules in one place and makes the training and testing code easier to read and reuse.

8. Why return the same position for an invalid move?
The agent does not pass through a wall or leave the grid. It remains in its current cell and receives a penalty.

9. Why is done False for an invalid move?
An invalid move does not reach the goal, so the episode continues.

10. Why use max(Q[next_position]) in the update?
Q-learning estimates the next state's best possible future action value, rather than the value of a randomly selected next action.

11. Why is the target only reward when done is True?
The goal ends the episode. There is no next decision after the terminal state, so no future Q-value is added.

12. What is temporal-difference error?
The difference between the learning target and the current Q estimate: target – Q(s,a). It indicates how much the estimate should be corrected.

13. What happens if alpha is 0?
Q-values never update, so the agent does not learn from experience.

14. What if alpha is very high, such as 1?
The new target replaces the old estimate completely at each update. Learning may become more sensitive to noisy or changing experiences.

15. What if gamma is 0?
The agent considers only immediate rewards and ignores future rewards.

16. What if gamma is close to 1?
Future rewards receive substantial weight. This can help in tasks where rewards are delayed, though learning may be more sensitive to long-term estimates.

17. Why are there 2,000 episodes?
Repeated experience gives the agent many chances to discover useful routes and refine Q-values. The chosen number is an experimental hyperparameter, not a universal requirement.

18. Why cap each episode at 100 moves?
To avoid unlimited wandering and keep training bounded.

19. Does the training guarantee the shortest path?
No. It can learn a useful route, but this small implementation does not mathematically guarantee the globally shortest path for every run or parameter setting.

20. What is a limitation of a Q-table?
It grows with the number of states and actions. It is practical for small discrete spaces but not for huge or continuous state spaces.

21. What is the difference between Q-learning and SARSA?
Q-learning is off-policy and uses the maximum next-state Q-value in its target. SARSA is on-policy and uses the Q-value of the next action actually selected.

22. What is the role of matplotlib?
It visualises the maze and the route. It is not responsible for training or updating Q-values.

23. Why use plt.show()?
It displays the generated figure in an interactive or notebook output window.

24. What if the printed result says Reached goal: False?
The greedy policy did not reach the goal within 50 test moves. Possible reasons include insufficient training, exploration/learning settings, or a learned loop.

25. How could you improve this experiment?
Track success rate and average steps, plot learning progress, tune alpha/gamma/epsilon decay, randomise tie-breaking, add a terminal-state check, and test across several seeds.

9. Practical exam quick revision

Remember this	One-line answer
State	Current maze coordinate (row, column).
Action	One of four movement directions.
Reward	-0.5 invalid, -0.1 normal move, +10 goal.
Q-table shape	(5, 5, 4).
Training	2,000 episodes; up to 100 moves each.
Testing	Greedy action selection; up to 50 moves.
Exploration	Random action with probability epsilon.
Exploitation	Choose action with maximum Q-value.
Update	Old Q + alpha × (target – old Q).

Explain the whole program in 30 seconds: “I represented a maze as a 5 × 5 NumPy array, where 1 means wall and 0 means free cell. The agent starts at (0,0) and must reach (4,4). The step function applies an action and returns the next state, reward, and terminal flag. I trained a Q-table for 2,000 episodes using epsilon-greedy exploration and the Q-learning update rule. Finally, I tested the greedy policy and used Matplotlib to plot the route.”

Accuracy note: The notebook's code is the source for the implementation details in this guide. The theory explanations are standard introductory reinforcement-learning concepts used to explain that implementation. The notebook does not report a guaranteed path, success-rate chart, or numerical training evaluation.